
Fictional Eureka

D2C Meets Fintech Reconciliation

AI-Powered Discrepancy Classifier & Smart Reconciliation Workflow

Product Requirements Document

Project Name	Fictional Eureka — D2C Meets Fintech Reconciliation
Organisation	Veltraxis Technologies Pvt. Ltd.
GitHub	github.com/10anshika/fictional-eureka
Document Date	June 2026
Version	1.0
Status	Engineering Review & Stakeholder Approval
Stack	FastAPI · Next.js 14 · Supabase · Celery · Redis · TypeScript
Distribution	Engineering, Finance, Founder, Product Leadership

1. Overview

1.1 What Is This Project?

Fictional Eureka is a **production-grade, full-stack financial reconciliation platform** built for Indian D2C (Direct-to-Consumer) and Quick Commerce brands. It solves a real, high-cost operational problem: when orders flow through Shopify or WooCommerce and settlements arrive via Razorpay, amounts rarely match exactly — because of platform commissions, GST, partial refunds, same-day returns, and timing delays. Finance teams today spend hours manually investigating each discrepancy.

This platform adds an **AI-powered discrepancy classification engine** on top of a working multi-platform reconciliation backend, a modern Next.js dashboard, enterprise-grade security (Supabase Row Level Security, JWT auth, Fernet-encrypted credentials), intelligent alerting (email, Slack, WhatsApp), Tally ERP export, and ITC recovery reporting — all deployable to Render with a single `render.yaml`.

1.2 Core Mission

To transform financial reconciliation from a manual, error-prone, time-intensive process into an intelligent, scalable, audit-ready workflow — minimising revenue leakage, cutting month-end close time, and letting finance teams focus on strategy rather than data triage.

1.3 What This PRD Covers

- **AI Discrepancy Classification Engine** — structured LLM prompt + fallback classifier, returning category, confidence score, explanation, and suggested action via `POST /api/ai/classify`
- **Multi-Platform Reconciliation Engine** — Shopify + WooCommerce order sync against Razorpay settlements; ghost order and refund trap detection
- **Database schema** — Supabase PostgreSQL with RLS, AI result columns, audit logs, and role-based access (Owner / Admin / CA / Viewer)
- **Dashboard** — Next.js 14 + TypeScript + Tailwind + Recharts; real-time transaction view, AI insights, classification filters
- **Alerting & exports** — configurable thresholds; email (Resend), Slack, WhatsApp; Tally ERP export; ITC/GST recovery reports
- **Deployment** — Render (Singapore region, closest to Mumbai); single `render.yaml`; health check endpoint at `/health`

2. Context & Background

2.1 The Indian D2C & Quick Commerce Landscape

India's D2C and quick commerce sector has grown explosively over the past 3–4 years. Platforms like Blinkit, Zepto, and BigBasket now promise 10-minute delivery, while thousands of D2C brands run storefronts on Shopify and WooCommerce. This growth is enabled by — and complicated by — marketplace proliferation, payment fragmentation, a multi-slab GST regime (5%, 12%, 18%, 28%), and evolving regulatory requirements around settlement accuracy and tax reporting.

The core problem: As D2C brands scale from **₹1–2 Cr.** to **₹50+ Cr.** GMV, manual reconciliation breaks down. Teams cannot feasibly match hundreds of orders per day against Razorpay settlement batches while

also handling partial refunds, same-day Quick Commerce returns (30–60 min return windows on Blinkit/Zepto), commission rate changes during promotional periods, and intra-state vs. interstate GST differences.

2.2 Why the Existing Approaches Fall Short

- **Spreadsheets:** Used by most brands under ₹20 Cr. GMV — does not scale, zero auditability, high error rate
- **Rule-based threshold alerts:** Flags variances above a fixed rupee value but cannot explain why a discrepancy occurred or what to do about it
- **International fintech platforms** (Razorpay Settlements, Stripe Financial Connections): focused on payment processing, not reconciliation intelligence; ignore Indian GST complexity and Quick Commerce return workflows

2.3 Why AI + a Full-Stack Platform Is the Right Answer

Deterministic rules alone cannot handle the full surface area of Indian D2C reconciliation. Fee structures change dynamically during sales events. Refund logic differs by product category. GST treatment depends on intra/inter-state routing. LLMs, prompted with domain-specific knowledge about Indian e-commerce fee structures and GST rules, can classify discrepancies with high confidence, generate human-readable explanations, and recommend actions — while the full platform handles ingestion, storage, alerting, and compliance.

3. What Is Actually Built (Verified from Repository)

3.1 Repository Structure

```
fictional-eureka/  
■■■■ backend/ # FastAPI application  
■ ■■■■ main.py # App entry point; registers all routes  
■ ■■■■ routes/  
■ ■ ■■■■ ai.py # POST /api/ai/classify endpoint  
■ ■■■■ requirements.txt  
■■■■ frontend/ # Next.js 14 + TypeScript application  
■■■■ database/ # Supabase PostgreSQL schema + migrations  
■■■■ docs/  
■ ■■■■ PRD_AI_Discrepancy_Classifier_Comprehensive.md  
■■■■ .github/workflows/ # CI/CD pipelines  
■■■■ render.yaml # One-click Render deployment config  
■■■■ TEST.md # Standalone curl test instructions  
■■■■ .python-version
```

3.2 Backend — FastAPI (Python)

Language breakdown: **Python 54.3%**, TypeScript 31.7%, PLpgSQL 13.7%, Other 0.3%

- **Framework:** FastAPI + Uvicorn
- **Database client:** Supabase Python SDK (PostgreSQL + Row Level Security)
- **HTTP client:** httpx (async)
- **Data validation:** Pydantic v2
- **Credential security:** Fernet symmetric encryption for stored API keys

- **Background jobs:** Celery + Redis
- **Email alerts:** Resend API
- **Error tracking:** Sentry DSN (configured via env var)
- **Platform integrations:** Shopify Admin API, WooCommerce REST API, Razorpay API
- **Deployment:** Render (Singapore region); `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`
- **Health check:** `GET /health`

3.3 Frontend — Next.js 14

- **Framework:** Next.js 14 + TypeScript + React
- **Styling:** Tailwind CSS + Radix UI + shadcn/ui components
- **Data client:** Supabase JS client
- **Charts:** Recharts (variance trends, classification breakdowns)
- **Auth:** Supabase Auth (JWT-based)

3.4 Database — Supabase PostgreSQL

- Row Level Security (RLS) enforced at database level
- Role-based access: Owner, Admin, CA (Chartered Accountant), Viewer
- AI result columns on `reconciled_transactions` table
- Audit logs — immutable; timestamp, `org_id`, input, output on every classification request
- PLpgSQL functions for reconciliation matching logic (13.7% of codebase)

3.5 Environment Variables (from `render.yaml`)

```
SUPABASE_URL # Supabase project URL
SUPABASE_SERVICE_ROLE_KEY # Service role key (server-side only)
SUPABASE_ANON_KEY # Public anon key
FERNET_KEY # Symmetric encryption key for credentials
CRON_SECRET # Secret for scheduled job authentication
RESEND_API_KEY # Email alert delivery (Resend)
RESEND_FROM_EMAIL # Sender address for alerts
RAZORPAY_BILLING_KEY_ID # Razorpay billing integration
RAZORPAY_BILLING_KEY_SECRET
SENTRY_DSN # Error tracking
APP_ENV # "production" / "development"
FRONTEND_URL # CORS origin
LOG_LEVEL # "INFO" in production
LLM_API_KEY # Optional – falls back to deterministic classifier if absent
```

4. Feature Specification

4.1 Feature 1: AI Discrepancy Classification Engine

The core feature. A production REST endpoint that accepts a discrepancy record and returns a structured AI classification. Crucially, **it works without an LLM API key** — falling back to a deterministic demo classification so the endpoint is always testable.

Endpoint

POST /api/ai/classify

Request Payload

```
{
  "order_id": "ORD12345",
  "platform": "Shopify",
  "expected_amount": 2500.00,
  "actual_amount": 2480.00,
  "difference": -20.00,
  "notes": "Possible commission rate change during sale"
}
```

Response Payload

```
{
  "classification": "Fee Mismatch",
  "confidence": 87,
  "explanation": "Shopify applied 2.5% commission (up from 2%) during sale period.",
  "suggested_action": "Acknowledge variance; no claim required.",
  "processed_at": "2026-06-20T09:22:34Z",
  "model_used": "claude-sonnet / fallback",
  "latency_ms": 1420
}
```

Database Write Behaviour

If a reconciled_transactions row exists with a matching shopify_order_id, the AI fields are automatically written to that row. Results are also verifiable via GET /api/transactions (authenticated) or directly in the Supabase dashboard.

Standalone Test (from TEST.md)

```
curl -X POST http://localhost:8000/api/ai/classify \
-H "Content-Type: application/json" \
-d '{
  "order_id": "ORD12345",
  "platform": "Shopify",
  "expected_amount": 2500.00,
  "actual_amount": 2480.00,
  "difference": -20.00,
  "notes": "Possible commission rate change during sale"
}'
```

Works without Supabase or LLM key — suitable for standalone demo.

Classification Categories

Category	Description
Fee Mismatch	Commission structure change; tier-based or promotional rate variation
Partial Return / Refund	Customer return, cancellation, partial fulfilment
Quick Commerce Same-Day Return	Blinkit/Zepto return within 30–60 min window; unique SLA
Timing Issue	Settlement batch delay, failed webhook, processing lag
GST Error	Tax slab mismatch; IGST vs. CGST/SGST routing error
Ghost Order	Order present in platform but absent from settlement

Category	Description
Refund Trap	Refund issued but not reflected in settlement correctly
Chargeback / Dispute	Customer dispute; chargeback fee applied by Razorpay
Platform Issue	Duplicate transaction; missing or duplicate settlement
Data Quality Issue	Missing order metadata; incomplete record
Other	Requires manual investigation; low-confidence classification

4.2 Feature 2: Multi-Platform Reconciliation Engine

The core matching engine ingests orders from Shopify (via Shopify Admin API) and WooCommerce (via WooCommerce REST API), then matches them against Razorpay settlement batches. Matching logic is implemented in PLpgSQL functions stored in Supabase.

- **Order sync:** Daily pull of Shopify and WooCommerce orders into reconciled_transactions
- **Settlement sync:** Daily pull of Razorpay settlement batches
- **Matching:** Deterministic rules (order ID + amount); PLpgSQL functions handle edge cases
- **Ghost order detection:** Orders present in platform data but absent from settlement
- **Refund trap detection:** Refunds issued but incorrectly (or not) reflected in settlement
- **Variance calculation:** Expected vs. actual amount; stored as difference field
- **AI trigger:** Discrepancies above threshold are automatically sent to POST /api/ai/classify

4.3 Feature 3: Database Schema (Supabase PostgreSQL)

AI Result Columns on reconciled_transactions

Column	Type	Purpose
ai_classification	TEXT	Classification category returned by AI
ai_confidence	FLOAT	Confidence score 0–100
ai_explanation	TEXT	Human-readable explanation of root cause
ai_suggested_action	TEXT	Recommended next step for finance team
ai_processed_at	TIMESTAMPTZ	Timestamp of AI classification
ai_reasoning_summary	TEXT	Extended reasoning from LLM (if available)
ai_model_used	TEXT	Model identifier or "fallback"
ai_latency_ms	INT	Round-trip latency in milliseconds

SQL — Indexes for Fast Dashboard Filtering

```
CREATE INDEX idx_ai_classification
ON reconciled_transactions(org_id, ai_classification);
CREATE INDEX idx_ai_confidence
ON reconciled_transactions(org_id, ai_confidence);
CREATE INDEX idx_ai_processed_at
```

```
ON reconciled_transactions(org_id, ai_processed_at);
```

Role-Based Access Control

Role	Access Level	Typical User
Owner	Full access — all orgs, settings, credentials	Founder / CTO
Admin	Full access within their organisation	Finance Head, Engineering Lead
CA	Read-only + resolution actions; no settings	Chartered Accountant, Auditor
Viewer	Read-only; no actions	Finance Analyst, Stakeholder

4.4 Feature 4: Pydantic Data Models

```
class DiscrepancyData(BaseModel):
    order_id: str
    platform: str # "Shopify" | "WooCommerce"
    expected_amount: float # in rupees
    actual_amount: float
    difference: float
    notes: str = ""
    transaction_type: str = "sale"
    metadata: Optional[dict] = None

class AIClassificationResult(BaseModel):
    classification: str
    confidence: float # 0-100
    explanation: str
    suggested_action: str
    reasoning_summary: str
    processed_at: datetime
    model_used: str
    latency_ms: int
```

4.5 Feature 5: Modern Dashboard (Next.js 14)

- **Transaction table:** Sortable, filterable by platform, status, classification, date range
- **AI classification badges:** Inline confidence scores and classification labels per row
- **Low-confidence queue:** Dedicated view for items with confidence <65% requiring manual review
- **Recovery opportunities:** Aggregated view of claim-eligible variances with total recoverable amount
- **Classification trend chart:** Recharts time-series of classification breakdown over 30 / 90 days
- **Ghost order panel:** Dedicated view for unmatched orders

4.6 Feature 6: Intelligent Alerting

Configurable threshold-based alerts with multi-channel delivery:

- **Ghost order alerts:** Triggered when an order is unmatched in settlement for >N hours
- **Variance threshold alerts:** Triggered when variance exceeds configured rupee amount
- **Low-confidence escalation:** High-value items with low AI confidence routed to Finance Manager
- **Channels:** Email (Resend API), Slack, WhatsApp

4.7 Feature 7: Export & Compliance

- **Tally ERP export:** Reconciled transactions exported in Tally-compatible format for accounting teams
- **ITC recovery report:** Identifies recoverable Input Tax Credit on Razorpay fees (18% GST on fees)
- **Monthly audit PDF + CSV:** Full classification summary; satisfies CA and tax authority requirements
- **Immutable audit log:** Every AI classification request logged with org_id, input, output, timestamp; cannot be modified

4.8 Feature 8: Deployment (Render)

```
# render.yaml (key settings)
services:
- type: web
  name: d2c-meets-fintech-reconciliation-backend
  env: python
  region: singapore # Closest to Mumbai for low latency
  plan: starter
  buildCommand: pip install -r backend/requirements.txt
  startCommand: uvicorn backend.main:app --host 0.0.0.0 --port $PORT
  healthCheckPath: /health
  autoDeploy: true
```

5. Out of Scope (Phase 1)

The following are explicitly deferred to Phase 2 or later:

- Custom fine-tuned LLM (current approach: structured prompting of general-purpose model)
- Multi-agent orchestration (LangGraph or similar)
- Automated claim filing with payment gateways
- Predictive variance prevention (ML-based forecasting of future discrepancies)
- A/B testing framework for prompt variants
- Webhook-based real-time classification (current: batch + on-demand)
- Amazon Seller Central and Flipkart integrations
- Mobile application

6. Personas & User Stories

6.1 Persona 1 — Rajesh Sharma, Finance Manager (D2C Brand, ₹25 Cr. GMV)

Profile: CA with 8+ years in finance ops. Responsible for month-end close, GST filing, and audit readiness. Currently spends ~50 hours/month on manual reconciliation.

Goals:

- Reduce reconciliation cycle from 5–7 days to 2–3 days
- Eliminate manual investigation for straightforward discrepancies
- Generate audit-ready documentation without extra effort
- Systematically identify and recover claimable variances

Key User Stories:

"As a Finance Manager, I want the system to automatically classify a ₹1,250 variance on a Shopify order, explain the likely cause (e.g., commission rate change on 15-May), and suggest an action — so I can resolve it in 30 seconds rather than investigating for 5 minutes."

"As a Finance Manager, I want to filter my dashboard to show only High-Confidence (>85%) items for quick sign-off, and Low-Confidence (<60%) items for manual review, so my team's time is used where it matters."

6.2 Persona 2 — Priya Desai, Reconciliation Analyst

Profile: BBA in Finance, 3–5 years in D2C ops. Handles daily order-settlement matching, flags discrepancies, and prepares monthly summaries.

Goals:

- Reduce per-item investigation time
- Understand patterns across discrepancy types to prevent recurrence
- Build domain expertise in e-commerce reconciliation

Key User Stories:

"As a Reconciliation Analyst, I want Low-Confidence variances automatically routed to a manual review queue so I can focus investigation time on genuinely ambiguous items, not ones the AI already resolved."

"As a Reconciliation Analyst, I want to see a trend chart showing how many Fee Mismatch classifications spike during Shopify sale periods, so I can anticipate volume spikes and propose preventive fee-tracking measures."

6.3 Persona 3 — Vikram Joshi, Engineering Lead

Profile: 8+ years full-stack; prior work at payment platforms. Owns API design, database architecture, and infrastructure reliability.

Goals:

- Integrate AI classifier without refactoring the core reconciliation engine
- Maintain complete, immutable audit trail for regulatory compliance
- Keep P95 classification latency under 5 seconds

Key User Stories:

"As an Engineering Lead, I want a clean `POST /api/ai/classify` endpoint that returns structured JSON so I can integrate AI classification into the pipeline with minimal changes to existing code."

"As an Engineering Lead, I want all LLM classification inputs and outputs logged with timestamps and `org_id` so we have a complete, immutable audit trail — and fallback behaviour when the LLM API is unavailable."

7. System Architecture & Data Flow

7.1 Architecture

```
Browser (Next.js 14 + TypeScript)
↓ HTTPS
Render (Singapore) — FastAPI Backend
  ■■ Reconciliation Engine ← Shopify Admin API
  ■ ← WooCommerce REST API
```

```

■ ← Razorpay API
■■ POST /api/ai/classify ← LLM API (optional)
■■ Alert Engine → Resend (email)
■ → Slack
■ → WhatsApp
■■ Export Engine → Tally ERP
→ ITC CSV
↓
Supabase (PostgreSQL + RLS)
■■ reconciled_transactions (AI result columns)
■■ audit_log (immutable)
■■ org_settings, thresholds
↓
Celery + Redis (background jobs: sync, batch classify)

```

7.2 AI Classification Data Flow

```

Reconciliation Engine detects variance
↓
Build DiscrepancyData (Pydantic)
↓
POST /api/ai/classify
■■ LLM_API_KEY present? YES → Call LLM with domain-specific system prompt
■ NO → Deterministic fallback classifier
↓
AIClassificationResult returned (classification, confidence, explanation,
suggested_action, latency_ms)
↓
Write AI fields to reconciled_transactions row (if order_id match found)
↓
Trigger downstream workflows:
■■ confidence < 65% + high value → manual review queue
■■ classification = "Claim Eligible" → recovery opportunities view
■■ classification = "Escalate" → alert to Finance Manager
↓
Dashboard reflects updated classification in real time

```

7.3 System Prompt — Engineered for Indian D2C Context

You are an expert financial reconciliation specialist for Indian D2C and Quick Commerce businesses. Classify discrepancies between expected and actual settlement amounts using the following domain knowledge:

PLATFORM FEES:

Shopify India: 2-2.5% commission (varies by category and promotion)
WooCommerce: Variable (plugin fees, payment gateway)
Razorpay: 2% + INR 3 standard; 1.99% subscriptions

TAX:

GST on fees: 18%
GST on goods: 5% / 12% / 18% / 28% depending on category
Intra-state: CGST + SGST; Inter-state: IGST

QUICK COMMERCE:

Blinkit / Zepto: same-day returns; 30-60 min return window
Different SLA and refund processing vs. standard D2C

Respond ONLY with JSON:
classification, confidence (0-100), explanation (1-2 sentences),
suggested_action

7.4 Edge Cases & Handling

Edge Case	Handling
Partial refund + fee mismatch on same order	AI classifies combined; explanation covers both root causes
Quick Commerce same-day return (Blinkit/Zepto)	Separate classification category; different SLA applied
Ghost order — in Shopify, missing from settlement	Detected by reconciliation engine; classified as "Ghost Order"
Refund trap — refund issued, not in settlement	Detected separately; classified as "Refund Trap"
Missing settlement data	Classification: "Data Quality Issue"; routed to manual queue
Duplicate transaction	Classification: "Platform Issue"; flagged for Razorpay follow-up
GST slab mismatch (IGST vs. CGST/SGST)	Classification: "GST Error"; explanation cites correct slab
LLM API unavailable	Fallback to deterministic rule-based classifier; no 500 error
LLM API key absent	Deterministic demo response returned; endpoint fully functional
LLM hallucination risk	Mitigated by structured JSON output format, confidence thresholds, analyst feedback loop

8. Non-Functional Requirements

Requirement	Target	Notes
P50 API Latency	<= 2 seconds	Measured at /api/ai/classify
P95 API Latency	<= 5 seconds	Monitored via backend logs
LLM Fallback	Always available	Deterministic classifier when LLM key absent or API down
System Availability	>= 99.5%	Render auto-deploy + health check at /health
Expected Classification Load	100–500 / day	Starter plan sufficient; scales horizontally
Database Retry	3 retries	Connection pooling with exponential backoff
Data Encryption at Rest	Yes	Supabase default + Fernet for stored credentials
Transport Security	TLS 1.3	Enforced by Render + Supabase
PII to LLM	Never	Customer names, emails, phone, card data excluded from prompts
Audit Log Retention	3 years minimum	Immutable; satisfies CA + tax authority requirements
Cost per Classification	<= \$0.50	General-purpose model; prompt caching where supported
Region	Singapore	Closest Render region to Mumbai; minimises latency for Indian users

9. Success Criteria & Metrics

9.1 Product Metrics

Metric	Target	How Measured
Auto-Classification Rate (confidence >75%)	80.75%	% of discrepancies classified without manual intervention
AI Classification Accuracy	80.85%	Manual spot-check of 100 randomly sampled classifications
Average Investigation Time (auto-classified)	4.2 min	Time from discrepancy flag to analyst resolution log
False Positive Rate	4.15%	Analyst feedback marking classifications as incorrect
API P95 Latency	4.5 sec	Logged in ai_latency_ms column
Revenue Recovery Identified	80.10% of variance	Total 8377; amount of claim-eligible classifications

9.2 Business Metrics (6-Month and 1-Year Targets)

Metric	6-Month Target	1-Year Target
Organisations Onboarded	5–10	30–50
Monthly Recurring Revenue	8377.75K–150K	8377.500K–1M
Customer Retention Rate	>90%	>95%
Net Promoter Score (NPS)	>40	>50

10. Test Plan

10.1 Standalone API Test (No Dependencies Required)

The AI classifier endpoint runs without Supabase or LLM key. From TEST.md:

```
# 1. Start backend
cd backend
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8000

# 2. Send test classification request
curl -X POST http://localhost:8000/api/ai/classify \
-H "Content-Type: application/json" \
-d '{"order_id":"ORD12345","platform":"Shopify",\
"expected_amount":2500.00,"actual_amount":2480.00,\
"difference":-20.00,\
"notes":"Possible commission rate change during sale"}'

# 3. Verify saved data (with auth)
GET /api/transactions
```

10.2 Test Suites

Suite	Scope	Key Test Cases
Unit — AI Classification	backend/routes/ai.py	Fee mismatch, partial return, timing issue, GST error, ghost order, fallback (no L
Unit — API Endpoints	FastAPI routes	Auth failure → 401, timeout handling → fallback, malformed payload → 422
Integration — DB Write	Supabase + reconciliation	AI result written to correct row; indexes return correct filtered results
Integration — Full Pipeline	End-to-end	Order synced → discrepancy detected → AI classified → dashboard updated
Performance	Latency + throughput	P95 <5s under 100 concurrent classification requests
UAT — Finance Manager	Dashboard	100% of discrepancies have classification; manual queue filters correctly
UAT — Audit Compliance	Audit log	Auditor can retrieve full decision trail for any classification by date range

11. Infrastructure & Deployment

11.1 Full Stack

Layer	Technology	Notes
Backend	FastAPI + Uvicorn	Python 3.x; async throughout
Frontend	Next.js 14 + TypeScript	Tailwind CSS + Radix UI + shadcn
Database	Supabase (PostgreSQL)	RLS + Auth + PLpgSQL functions
Object Storage	Supabase Storage	Audit exports, generated reports
Background Jobs	Celery + Redis	Order sync, batch classification, scheduled exports
Email Alerts	Resend API	Transactional; alert on thresholds
Error Tracking	Sentry	DSN configured via env var
CI/CD	GitHub Actions + Render	Auto-deploy on push to main; .github/workflows/
Charts	Recharts	Used in frontend dashboard
HTTP Client	httpx	Async; used for platform API calls

11.2 Monthly Infrastructure Cost Estimate

Service	Estimated Cost
Supabase (Pro plan)	Ź10K–15K
Render (FastAPI hosting, Singapore)	Ź2K–5K
LLM API (per-call)	Ź1K–2K
Redis (Celery broker)	Ź1K–2K

Service	Estimated Cost
Resend (email alerts)	₹500–1K
Sentry	₹0–1K
Total	₹15K–26K / month

12. Business Model & Product Strategy

12.1 Market Opportunity

- Indian D2C and Quick Commerce sector growing 30%+ annually
- No purpose-built, AI-assisted reconciliation tool for the Indian D2C market exists today
- Finance teams at █10–200 Cr. GMV brands have budget and genuine pain — not price-sensitive
- Tally ERP export and ITC recovery are non-negotiable for Indian compliance — strong lock-in once integrated

12.2 Target Customer Tiers

Tier	GMV Range	Willingness to Pay	Key Pain
Starter — Early Adopters	₹5–20 Cr.	₹10–20K/month	Manual reconciliation consuming >40 hrs/month
Growth	₹20–100 Cr.	₹30–75K/month	Audit pressure; multiple platforms; team scaling cost
Enterprise	₹100+ Cr.	₹150K+/month	GST compliance risk; board-level audit readiness

12.3 Proposed Pricing

Plan	GMV	Classifications / Month	Price
Starter	<₹20 Cr.	Up to 10K	₹15K
Growth	₹20–100 Cr.	Up to 50K	₹50K
Enterprise	>₹100 Cr.	Unlimited	₹150K+

13. Risks & Mitigation

Risk	Probability	Impact	Mitigation
LLM API unavailable	Medium	Low	Deterministic fallback always active; no 500 errors
AI misclassification	Medium	Medium	Confidence thresholds; low-confidence routed to manual queue; an
LLM cost overrun	Medium	Low	Per-call cost monitoring; prompt caching; batch off-peak
GST rule change	Low	High	System prompt updated by PM; no code change required

Risk	Probability	Impact	Mitigation
Supabase downtime	Low	High	Connection pooling + 3 retries; Render health check restarts service
Data privacy breach	Low	Critical	No PII in LLM prompts; Fernet encryption; Supabase RLS; TLS 1.3
Shopify / Razorpay API change	Medium	Medium	Versioned API clients; integration tests in CI/CD
Feature creep beyond Phase 1	High	Medium	Strict MVP scope; Phase 2 backlog maintained separately

14. Timeline (12-Week MVP)

Week	Milestone	Key Deliverables
W1–W2	Design & Planning	PRD finalised, API spec, DB schema, render.yaml skeleton
W3–W4	Backend Foundation	FastAPI scaffold, Supabase schema + RLS, Shopify/Razorpay ingestion
W5–W6	AI Classifier	POST /api/ai/classify, system prompt, fallback classifier, DB write
W7–W8	Full Reconciliation Pipeline	Ghost order + refund trap detection, alert engine, Celery jobs
W9–W10	Frontend Dashboard	Transaction table, AI badges, low-confidence queue, Recharts trends
W11	Exports & Compliance	Tally ERP export, ITC recovery report, audit PDF/CSV
W12	Testing & Go-Live	Unit + integration + UAT; Render deployment; TEST.md validated

15. Glossary

Term	Definition
Discrepancy	Variance between expected and actual settlement amount on a given order
Settlement	Payment transferred by Razorpay to the seller after deducting fees
Ghost Order	Order present in platform data (Shopify/WooCommerce) but absent from settlement batch
Refund Trap	Refund issued to customer but not correctly reflected in settlement
ITC	Input Tax Credit — GST paid on Razorpay fees, recoverable by GST-registered businesses
RLS	Row Level Security — Supabase/PostgreSQL feature enforcing per-row access control
Confidence Score	AI's self-reported probability (0–100) that its classification is correct
Fallback Classifier	Deterministic rule-based classifier used when LLM API key is absent or API is down
Tally ERP	Accounting software widely used by Indian SMBs; export format required for bookkeeping
Quick Commerce	Ultra-fast delivery platforms (Blinkit, Zepto) with 10-30 min delivery windows
CGST / SGST / IGST	Central / State / Integrated GST; routing depends on intra- vs. inter-state transaction

16. Approval & Sign-Off

Prepared by: Product Manager, Veltraxis Technologies Pvt. Ltd.

Date: June 2026

Status: Ready for Engineering Review

Sign-offs required:

- **Product Lead** — Confirms alignment with product strategy and Phase 1 scope
- **Engineering Lead** — Confirms technical feasibility, stack choices, and timeline
- **Founder** — Confirms business case and prioritisation
- **Finance Stakeholder** — Confirms market fit, persona accuracy, and compliance coverage

Version History

Version	Date	Author	Summary
1.0	20-Jun-2026	Product Team	Initial PRD — written against actual repository (github.com/10anshika/fictional-